ExamsIndex
Practice Materials & Tests

DEMO VERSION

Python Institute

PCPP-32-101 Exam

PCPP1 – Certified Professional in Python Programming 1

Exam Latest Version: 6.0

DEMO VERSION

Question 1. (Single Select)

Select the true statement about composition

A: Composition extends a class's capabilities by adding new components and modifying the existing ones.
B: Composition allows a class to be projected as a container of different classes
C: Composition is a concept that promotes code reusability while inheritance promotes encapsulation.
D: Composition is based on the has a relation: so it cannot be used together with inheritance.

Correct Answer: B

Explanation:

Composition is an object-oriented design concept that models a has-a relationship. In composition, a class known as composite contains an object of another class known as component. In other words, a composite class has a component of another class1.

1. Composition allows a class to be projected as a container of different classes.

Composition is a concept in Python that allows for building complex objects out of simpler objects, by aggregating one or more objects of another class as attributes. The objects that are aggregated are generally considered to be parts of the whole object, and the containing object is often viewed as a container for the smaller objects.

In composition, objects are combined in a way that allows for greater flexibility and modifiability than what inheritance can offer. With composition, it is possible to create new objects by combining existing objects, by using a container object to host other objects. By contrast, with inheritance, new objects extend the behavior of their parent classes, and are limited by that inheritance hierarchy.

Official Python documentation on Composition: https://docs.python.org/3/tutorial/classes.html#composition

GeeksforGeeks article on Composition vs Inheritance: https://www.geeksforgeeks.org/composition-vs-inheritance-python/

Real Python article on Composition and

DEMO VERSION

Inheritance: https://realpython.com/inheritance-composition-python/

Question 2. (Multi Select)

What is a___traceback___?

(Select two answers)

A: An attribute owned by every exception object
B: A special method delivered by the traceback module to retrieve a full list of strings describing the traceback
C: An attribute that is added to every object when the traceback module is imported
D: An attribute that holds interesting information that is particularly useful when the programmer wants to store exception details in other objects

Correct Answer: A, D

Explanation:

The correct answers are A. An attribute owned by every exception object and D. An attribute that holds interesting information that is particularly useful when the programmer wants to store exception details in other objects. A traceback is an attribute of an exception object that contains a stack trace representing the call stack at the point where the exception was raised. The traceback attribute holds information about the sequence of function calls that led to the exception, which can be useful for debugging and error reporting.

Question 3. (Single Select)

Analyze the following snippet and select the statement that best describes it.

DEMO VERSION

```
def f1(*arg, **args):
    pass
```

A: The code is syntactically correct despite the fact that the names of the function parameters do not follow the naming convention
B: The *arg parameter holds a list of unnamed parameters
C: The code is missing a placeholder for unnamed parameters.
D: The code is syntactically incorrect - the function should be defined as def f1 (*args, **kwargs) :

Correct Answer: B

Explanation:

The provided code snippet defines a function f1 that accepts variable-length arguments using the *args and **kwargs syntax. The *args parameter allows for an arbitrary number of unnamed arguments to be passed to the function as a tuple, while the **kwargs parameter allows for an arbitrary number of named arguments to be passed to the function as a dictionary.

Therefore, the correct statement that best describes the code is:

1. The *args parameter holds a list of unnamed parameters, while the **kwargs parameter holds a dictionary of named parameters.

Official Python documentation on Function definitions: https://docs.python.org/3/tutorial/controlflow.html#defining-functions

The arg parameter holds a list of unnamed parameters. In the given code snippet, the f1 function takes two arguments: *arg and **kwarg. The *arg syntax in the function signature is used to pass a variable number of non-keyword (positional) arguments to the function. Inside the function, arg is a tuple containing the positional arguments passed to the function. The **kwarg syntax in the function signature is used to pass a variable number of keyword arguments to the function. Inside the function, kwarg is a dictionary containing the keyword arguments passed to the function.

DEMO VERSION

Question 4. (Single Select)

Analyze the following snippet and decide whether the code is correct and/or which method should be distinguished as a class method.

```
class Crossword:
    number_of_Crosswords = 0

    def __init__(self, height, width):
        self.height = height
        self.width = width
        self.progress = 0

    @staticmethod
    def isElementCorrect(word):
        if self.isSolved():
            print('The crossword is already solved')
            return True
        result = True
        for char in word:
            if char.isdigit():
                result = False
                break
        return result

    def isSolved(self):
        if self.progress == 100:
            return True
        return False

    def getNumberOfCrosswords(cls):
        return cls.number_of_Crosswords
```

A: There is only one initializer, so there is no need for a class method.
B: The getNumberofCrosswords () method should be decorated With @classmethod.
C: The code is erroneous.
D: The gexNumberOfcrosswords () and issrived methods should be decorated with @classzoechod.

Correct Answer: B

Explanation:

The correct answer is B. The getNumberofCrosswords() method should be decorated with

DEMO VERSION

@classmethod. In the given code snippet, the getNumberofCrosswords method is intended to be a class method that returns the value of the numberofcrosswords class variable. However, the method is not decorated with the @classmethod decorator and does not take a cls parameter representing the class itself. To make getNumberofCrosswords a proper class method, it should be decorated with @classmethod and take a cls parameter as its first argument.

1. The getNumberofCrosswords() method should be decorated with @classmethod.

This is because the getNumberofCrosswords() method is intended to access the class-level variable numberofcrosswords, but it is defined as an instance method, which requires an instance of the class to be created before it can be called. To make it work as a class-level method, you can define it as a class method by adding the @classmethod decorator to the function.

Here's an example of how to define getNumberofCrosswords() as a class method:

```
class Crossword:

numberofcrosswords = 0

def __init__(self, author, title):

self.author = author

self.title = title

Crossword.numberofcrosswords += 1

@classmethod

def getNumberofCrosswords(cls):

return cls.numberofcrosswords
```

In this example, getNumberofCrosswords() is defined as a class method using the @classmethod decorator, and the cls parameter is used to access the class-level variable numberofcrosswords.

Official Python documentation on Classes: https://docs.python.org/3/tutorial/classes.html

DEMO VERSION

Question 5. (Single Select)

Analyze the code and choose the best statement that describes it.

```
class Item:
    def __init__(self, initial_value):
        self.value = initial_value

    def __ne__(self, other):
        ...
```

A: ___ne___() is not a built-in special method
B: The code is erroneous
C: The code is responsible for the support of the negation operator e.g. a = - a.
D: The code is responsible for the support of the inequality operator i.e. i =

Correct Answer: D

Explanation:

The correct answer is D. The code is responsible for the support of the inequality operator i.e. i != j. In the given code snippet, the __ne__ method is a special method that overrides the behavior of the inequality operator != for instances of the MyClass class. When the inequality operator is used to compare two instances of MyClass, the __ne__ method is called to determine whether the two instances are unequal.

DEMO VERSION

ExamsIndex

Demo PDF Complete

Your PCPP-32-101 Demo (5 Questions)

Get the Complete Version

Full Questions with Detailed Explanations

Interactive Web-Based Exams Available

To get 50% off, use Coupon Code: OCT50

https://examsindex.com/exam/pcpp-32-101